```
NAME                                READY   STATUS    RESTARTS   AGE
catalog-service-6c5fc7b955-9kvgf    1/1     Running   0          3m94s
catalog-service-6c5fc7b955-n7rgl    1/1     Running   0          3m94s
```

Kubernetes is designed to ensure the availability of each application. If enough resources are available, it will try to deploy the two replicas on two different nodes. If one node crashes, the application will still be available on the other one. At the same time, Kubernetes takes care of deploying the second instance somewhere else to ensure there are always two replicas up and running. You can check which node each Pod has been allocated on with `kubectl get pod -o wide`. In our case, the minikube cluster has only one node, so both instances will be deployed together.

If you're curious, you can also try to update the application-staging.yml file, apply the `Kustomization` to the cluster again (`kubectl apply -k .`), and see how the Catalog Service Pods are restarted one after the other (*rolling restarts*) to load the new ConfigMap with zero downtime. To visualize the sequence of events, you can either use Octant or launch this command on a separate Terminal window before applying the `Kustomization`: `kubectl get pods -l app=catalog-service --watch`.

When you're done testing the application, you can terminate the port-forwarding process with Ctrl-C and delete the cluster with `./destroy-cluster.sh` from polar-deployment/kubernetes/platform/development.

Now that you've learned the fundamentals of configuring and deploying Spring Boot applications in Kustomize, it's time to go to production. That's what the next chapter is all about.

Polar Labs